Individual Report

PE6203 Group Project 2 — Safety Approval Policies for Agent Harnesses

Thummalapally Keerthi Reddy · keerthir001@e.ntu.edu.sg · Project: *Evidence-Grounded Approval Gates for Tool-Using Agents*

1. Individual Contributions

My primary responsibility was the **simulated human reviewer**, the component the gate calls whenever the policy escalates a tool call. I also built the logic for a **custom invoice evaluation suite**, and contributed to the **failure and escalation-value analysis** and the project's documentation.

1.1 Simulated human reviewer

I designed and implemented the LLM-based reviewer that stands in for a human approver, since real reviewers cannot scale to a benchmark of over a thousand runs.

- **Four-field reviewer view.** The reviewer sees only what a real person approving the call would see: the user's request, the proposed tool, its arguments, and the latest tool output (truncated to 1,500 characters). Its prompt treats tool output strictly as data, not as instructions.
- **Isolation from oracle information.** The reviewer never has access to benchmark labels, attack annotations, or hidden environment state. A leakage test confirms none of this information can reach the reviewer's prompt, keeping the evaluation tamper-evident.
- **Prompt refinement (v1 to v2).** Version 1 wrongly rejected a legitimate refund whose amount the agent had correctly filled in from the transaction history. I diagnosed this and revised the prompt so version 2 explicitly allows details to be taken from visible data, while preserving attack resistance.
- **Fail-closed behaviour.** Only a well-formed JSON `APPROVE` executes a call; any malformed output or provider error defaults to `DENY`. The model runs at temperature 0 for deterministic decisions.
- **Model adapters and metering.** I wrote adapters for OpenAI and OpenRouter and the per-run record format, with token metering so the reviewer's cost could be reported separately (about US$0.003 to US$0.004 per 160 runs under our policy).

1.2 Custom invoice evaluation suite (logic)

Beyond AgentDojo's banking tasks, I built the logic for an independent invoice-payment suite with a trusted **vendor registry**, including the difficult "right vendor, wrong amount" case the benchmark does not cover. This logic underpinned the transfer experiment, where the same gate and policy, unchanged, paid the invoice at every budget with 0 of 9 attacks succeeding. The work also surfaced the **forged-document gap**: if an attacker silently rewrites the IBAN on a bill, the grant points at the attacker, and an independent payee source such as the vendor registry is what closes it.

1.3 Analysis and documentation

I contributed to classifying failures into over-permissive, over-conservative and reviewer-induced types, and to interpreting the representative trajectories. I also contributed to the escalation-value discussion of which call types genuinely benefit from human review. Alongside this, I helped prepare draft copies of the project's written report and presentation materials, bringing the team's findings, results and limitations into a consistent narrative.

2. Lessons Learned

A limited view is what makes a simulated reviewer trustworthy. It was tempting to give the reviewer more context to improve its decisions. Restricting it to what a human would actually see, and proving that restriction with a test, is what lets us claim the results are not inflated by oracle access. The leakage test mattered as much as the reviewer itself.

Prompt wording changes behaviour in subtle, measurable ways. The v1 reviewer looked reasonable but silently rejected a valid refund because it treated agent-filled details as unauthorized. The fix was one clarifying sentence. The lesson is that a reviewer must be calibrated on both safe and unsafe cases before it is trusted to judge a policy, not assumed correct from the prompt alone.

More human review is not more safety. This surprised me most. Full escalate spent up to 279 reviewer calls and never rose above the 0.31 completion floor, while our reviewer was called at most 51 times and never approved an attack. Safety came from the authorization logic deciding which calls were genuinely ambiguous, not from the volume of reviews.

Fail-closed accounting keeps results honest. Because an API error looks identical to a strict reviewer, I logged reviewer errors separately from genuine denials. Without this, a provider failure could have masqueraded as safe behaviour.

The reviewer's main weakness was a lack of context, not capability. In clean tasks 9 and 10 the reviewer approved incomplete writes because it did not know why the call had been escalated. A correct decision needed the escalation reason, not a stronger model.

On a personal level, this was my first time building on a large external codebase and grounding the work in research papers. Plugging our gate into AgentDojo meant reading an unfamiliar framework closely enough to find the one place to intervene, rather than rewriting it, and reading the prompt-injection literature taught me to frame our design against what had already been tried and to borrow the field's vocabulary and evaluation methods. I came away more comfortable working inside systems and research I did not write, which is how most real development and research actually happens.

3. Future Plans

If this work continues, I would focus on the parts closest to my contributions:

- **Give the reviewer the escalation reason.** The two reviewer-induced failures (tasks 9 and 10) both came from the reviewer not knowing why a call was escalated. Passing the triggering rule into its context is a small change that should directly remove these failures.

- **Close the forged-document gap with payee corroboration.** The invoice suite already showed a vendor registry defends against a silently rewritten IBAN. Extending this to the banking suite, by checking payees against an independent trusted source rather than the document alone, would address the one gap AgentDojo does not test.
- **Broaden the reviewer's robustness.** I would test the reviewer against adaptive attacks aimed at its own prompt, and measure its stability across models and across repeated runs, since our current results use a single run per setting with no confidence intervals.
- **Add a reviewer confidence score (my own direction).** Rather than a bare APPROVE or DENY, I would have the reviewer return a confidence level, so that low-confidence approvals could be held back or routed differently instead of executing silently, as happened in tasks 9 and 10.
- **Build a small human-labelled calibration set (my own direction).** I would collect a handful of hand-labelled escalation cases, safe and unsafe, and use them as a fixed check that any future reviewer prompt or model must pass before it is used for evaluation, turning prompt tuning from a judgement call into a measured one.